

SESSION 2 — PROBLEM SOLVING & SEARCH

CMS707

Artificial Intelligence

-  **Course:** CMS707 — Artificial Intelligence
-  **Lecturer:** Dr. Friday Orji
-  **Institution:** University of Science & Technology
-  **Content:** Session 2 Exercises — Full Worked Answers

Prepared by

Mbosinwa Awunor · PGD Computer Science

Problem Formulation

State Space

BFS Search

Missionaries & Cannibals

Map Colouring

EXERCISE 1

Complete Problem Formulations — Overview & Q1a

Task: Give a complete problem formulation for each scenario, precise enough to be implemented.

Framework (Dr. Orji's 4 components): State · Initial State · Goal State · Actions

QUESTION 1A — FOUR-COLOUR MAP COLOURING

Colour a planar map using only **4 colours** such that no two adjacent regions share the same colour.

COMPONENT	FORMULATION
State	A partial or complete assignment of colours from {Red, Blue, Green, Yellow} to each region on the map
Initial State	All regions uncoloured — empty assignment {}
Goal State	Every region has a colour AND no two adjacent regions share the same colour
Actions	Assign one of {Red, Blue, Green, Yellow} to one uncoloured region, provided it does not conflict with any already-coloured adjacent region
Path Cost	Not relevant — any valid complete colouring is a solution

Key insight: This is a *Constraint Satisfaction Problem (CSP)*. The Four-Colour Theorem (proved 1976) guarantees a solution always exists for any planar map. Backtracking search with constraint propagation is the optimal algorithm.

Monkey and Bananas — Problem Formulation

A 3-foot monkey is in a room. Bananas hang 8 feet high. A moveable chair is available. How does the monkey get the bananas?

COMPONENT	FORMULATION
State	A 4-tuple: (monkey_pos, on_chair?, chair_pos, has_bananas?) monkey_pos = floor location on_chair? ∈ {T, F} chair_pos = chair floor location has_bananas? ∈ {T, F}
Initial State	(pos_A, False, pos_B, False) — monkey and chair at different positions, bananas not yet reached
Goal State	(_, _, _, True) — has_bananas = True
Actions	Walk(to) — monkey moves along floor to new position Push(chair, to) — monkey (on floor) pushes chair to new position ClimbUp — monkey climbs chair (only if monkey_pos = chair_pos) ClimbDown — monkey gets off chair Grasp — grab bananas (only if: on_chair=True AND chair_pos = under_bananas)

★ OPTIMAL SOLUTION — 4 ACTIONS

1. Walk to chair position
2. Push chair to under bananas
3. ClimbUp
4. Grasp → **Goal reached!**

WHY THIS FORMULATION?

We only include distinctions that affect goal reachability. We do not track exact floor coordinates — only relative positions (monkey at chair? chair under bananas?). This keeps the state space minimal and implementable.

Illegal Record & Three Jugs Problem

QUESTION 1C — FINDING THE ILLEGAL INPUT RECORD

A program outputs "Illegal input record" for a file. Records are **independent**. Find the illegal record.

COMPONENT	FORMULATION
State	A subset S of input records currently under test
Initial State	S = the full set of all input records
Goal State	S is a minimal subset — removing any record causes the error to disappear (isolates the culprit)
Actions	Split(S) — divide into halves; Test(S) — run program on S ; Discard(half) — drop the half that doesn't trigger the error

Algorithm — Binary Search: Since records are independent, test first half → if error, recurse into first half; else recurse into second half. **$O(\log n)$** program runs. Without independence: exponential 2^n subsets.

QUESTION 1D — THREE JUGS: MEASURE EXACTLY 1 GALLON

COMPONENT	FORMULATION
State	Triple (x, y, z) — litres in 12-gal, 8-gal, 1-gal jugs. $x \in \{0 \dots 12\}$, $y \in \{0 \dots 8\}$, $z \in \{0, 1\}$
Initial State	$(12, 0, 0)$ — 12-gal full, others empty
Goal State	$z=1$, OR $(x=1, y=0, z=0)$, OR $(y=1, x=0, z=0)$ — exactly 1 gallon isolated
Actions	Fill(jug) from tap; Empty(jug) to ground; Pour(A→B) — pour until A empty or B full

Key observation: Fill the 1-gallon jug from the 12-gallon jug → state $(11, 0, 1)$. Goal reached in **1 action**. Good formulation reveals the simplest solution immediately.

Missionaries & Cannibals — Precise Formulation

3 missionaries + 3 cannibals on left bank. Boat holds **1 or 2 people**. Missionaries must **never be outnumbered** by cannibals on *either* bank (when missionaries are present).

STATE REPRESENTATION

(M_L, C_L, Boat)

M_L = missionaries on left (0–3)

C_L = cannibals on left (0–3)

Boat ∈ {L, R}

Right bank: M_R = 3–M_L, C_R = 3–C_L

START & GOAL

Initial: (3, 3, L) — all on left

Goal: (0, 0, R) — all on right

VALIDITY CONSTRAINT

State is UNSAFE if missionaries present but outnumbered:

Left: M_L > 0 AND M_L < C_L → **✗**

Right: M_R > 0 AND M_R < C_R → **✗**

VALID (M_L, C_L) COMBINATIONS

M_L	C_L	LEFT	RIGHT (3–M, 3–C)	
0	0,1,2,3	✓ 0M	✓	VALID
1	1	✓ 1≥1	✓ 2≥2	VALID
2	2	✓ 2≥2	✓ 1≥1	VALID
3	0,1,2,3	✓ 3≥all	✓ 0M	VALID
1	2 or 3	✗ 1<2	—	INVALID
2	1 or 3	—	✗ 1<2	INVALID

10 valid (M_L, C_L) pairs × 2 boat positions = 20 valid states maximum in the entire state space.

Missionaries & Cannibals — State Space Diagram

COMPLETE REACHABLE STATE SPACE (M_L, C_L, BOAT) — FORMAT: CROSSINGS SHOWN AS →RIGHT / ←LEFT



BFS Optimal Solution — 11 Crossings

Algorithm: BFS (FIFO Queue)

Guarantees shortest path. Explores states level by level from (3,3,L) until (0,0,R) found.

Valid crossing actions: 1M · 2M · 1C · 2C · 1M+1C

Each must result in a safe state on BOTH banks.

#	DIR	WHO CROSSES	STATE AFTER (M_L, C_L, BOAT)	LEFT BANK	RIGHT BANK	
Start	—	—	(3, 3, L)	3M 3C	0M 0C	✓
1	→	2 Cannibals	(3, 1, R)	3M 1C	0M 2C	✓
2	←	1 Cannibal	(3, 2, L)	3M 2C	0M 1C	✓
3	→	2 Cannibals	(3, 0, R)	3M 0C	0M 3C	✓
4	←	1 Cannibal	(3, 1, L)	3M 1C	0M 2C	✓
5	→	2 Missionaries	(1, 1, R)	1M 1C	2M 2C	✓
6 ⚠	←	1M + 1C back	(2, 2, L)	2M 2C	1M 1C	✓
7	→	2 Missionaries	(0, 2, R)	0M 2C	3M 1C	✓
8	←	1 Cannibal	(0, 3, L)	0M 3C	3M 0C	✓
9	→	2 Cannibals	(0, 1, R)	0M 1C	3M 2C	✓
10	←	1 Cannibal	(0, 2, L)	0M 2C	3M 1C	✓
11 ✓	→	2 Cannibals	(0, 0, R)	0M 0C	3M 3C	✓ GOAL

⚠ **Step 6 — The counterintuitive move:** Bringing 1M+1C *back* to the left feels wrong but is the only move that avoids leaving missionaries outnumbered at (1,1,R). This is why Dr. Orji emphasises *analysing* the problem before searching.

Repeated States & Algorithm Analysis

Should we check for repeated states? **YES — absolutely essential.**

WITHOUT REPEATED-STATE CHECKING

BFS cycles infinitely:

(3,3,L) → 2C right → (3,1,R)
 → 1C back → (3,2,L)
 → 2C right → (3,0,R)
 → 1C back → (3,1,L)
 → ... → loops back → **NEVER TERMINATES**

WITH REPEATED-STATE CHECKING

BFS maintains a reached set.
 Any state already in reached is discarded.
 Only 20 valid states in the space.
 Terminates in **O(20) state expansions**.
 Finds optimal 11-crossing solution. ✓

WHY BFS IS THE RIGHT ALGORITHM

- Guarantees **shortest path** (fewest crossings)
- Complete — will always find the solution if one exists
- State space is small (20 states) — memory not an issue
- No heuristic needed — we want optimal, not just any solution

WHY NOT DFS?

- DFS could get trapped in cycles
- DFS does not guarantee shortest path
- Without depth limit, DFS may loop forever
- BFS is clearly better here given the small state space

COMPLEXITY

Valid states: **20**

Optimal path length: **11 crossings**

Two symmetric solutions exist (swapping the cannibal-first and missionary-first phases)

Dr. Orji's connection: This problem — like the Farmer's Problem — demonstrates that *checking repeated states is not optional*. Both problems have cyclic state spaces. Without the reached set, no search terminates correctly.

SESSION 2 — SUMMARY

EXERCISE 1A — MAP COLOURING

CSP formulation. 4 colours, constraint on adjacency. Backtracking + constraint propagation.

EXERCISE 2A — M&C FORMULATION

State (M_L, C_L, Boat). 10 valid (M,C) pairs. 20 total valid states. Safety constraint on both banks.

EXERCISE 1B — MONKEY & BANANAS

4-tuple state. Goal-based agent. Optimal in 4 actions: Walk → Push → ClimbUp → Grasp.

EXERCISE 2A — STATE SPACE

Complete state space mapped and verified. All 20 valid states identified and connected.

EXERCISE 1C — ILLEGAL RECORD

Binary search formulation. Independent records → $O(\log n)$ program runs to isolate the culprit.

EXERCISE 2B — BFS SOLUTION

11 crossings optimal. Step 6 (bring 1M+1C back) is the counterintuitive key move.

EXERCISE 1D — THREE JUGS

Triple (x,y,z) state. Trivial 1-action solution: fill the 1-gal jug from the 12-gal jug.

EXERCISE 2B — REPEATED STATES

Essential. Without checking, BFS cycles infinitely. With reached set: $O(20)$ termination.

PREPARED BY

Mbosinwa Awunor

PGD Computer Science · Session 2 Exercises
Dr. Friday Orji · University of Science & Technology